

β

This project is currently under beta-test, until it is validated by a fair number of students or a decent delay has elapsed.

- *Please report any typo, incoherence, inconsistency, error, using the form <https://tally.so/r/3lVDJo>.*
- *A dedicated Slack channel exists to exchange on beta-test projects: [#_world_beta_test](#).*
- *A FAQ page dedicated to beta-test projects is available [here](#).*
- *The project currently awards 0 XP, on purpose; XP will be granted once the beta-test is over and conclusive.*





Inception-of-Wisdom (IoW)

Summary: This document is a System Administration and Artificial Intelligence related exercise.

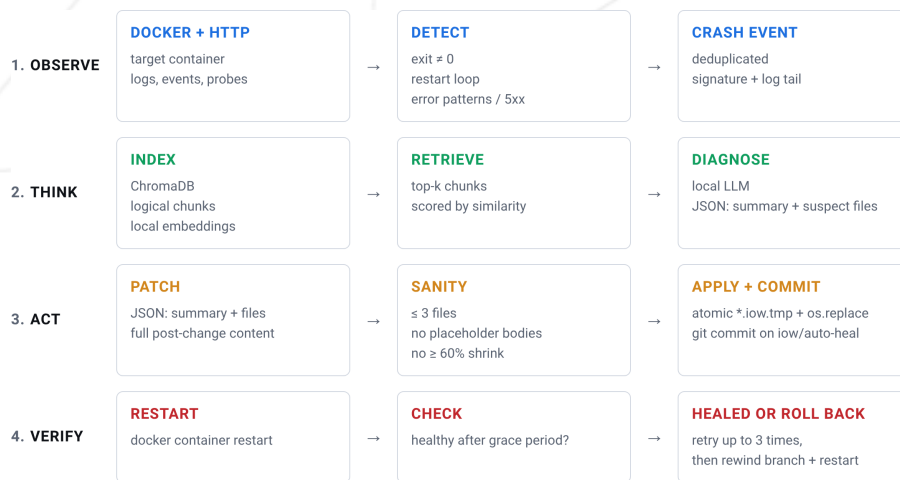
Version: 1.1

Contents

I	Preamble	2
II	Foreword	3
III	Introduction	4
IV	General guidelines	5
V	Mandatory part	7
V.1	Part 1: Observer	7
V.2	Part 2: Analyst	9
V.3	Part 3: Wisdom Loop	11
VI	Bonus part	14
VII	Submission and peer-evaluation	15

Chapter I

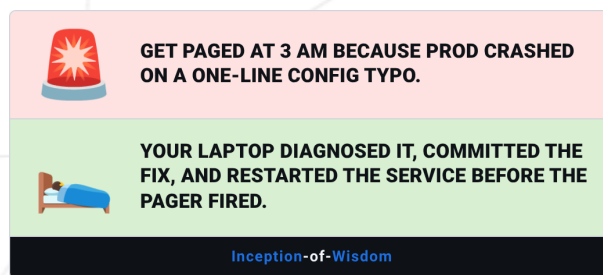
Preamble



The four beats of an autonomous loop: observe the running service, think about what is wrong, act on the code, verify that the fix took.

Chapter II

Foreword



There is a recurring fantasy in operations: a service that fixes itself. The pager never goes off because the system has already noticed, understood, and patched the thing that would have woken you up. For years this lived in vendor pitch decks. The pieces to actually build it now fit on a laptop.

You already have most of them. *Inception-of-Things* taught you how a Git push reshapes a running cluster. *Inception-of-Context* taught you how a small local model reads a codebase and proposes a patch that does not break it. This project wires the two ideas into a single closed loop.

The agent watches a real service, notices when something is wrong, retrieves the code that probably caused it, asks the model what to change, applies the patch, commits it, restarts the service, and checks that the symptom is gone. If the fix does not take, the loop rolls back as if the agent never spoke up.

Chapter III

Introduction

This project is the autonomous version of what you built in *Inception-of-Context*: instead of waiting for a human intent, the agent reads its own operational state and decides when to act. The loop has four beats. **Observation** pulls container health and log lines from the Docker daemon. **Thinking** retrieves the relevant code and configuration chunks from a local index and asks a small local LLM for a diagnosis. **Action** produces a structured patch, applies it atomically, and commits the change to Git. **Verification** restarts the target and checks whether the failure signal disappeared, rolling everything back through Git if it did not.

Everything must run **locally**. Any call to a remote LLM service is forbidden and results in a grade of zero. The agent itself runs in a container, but it has to be able to inspect and restart its sibling containers and to write to the target's repository on disk.



This project assumes you have done both *Inception-of-Things* and *Inception-of-Context*. IoT gave you the deployment substrate, IoC gave you the retrieval and patching substrate, and this one snaps them together. Re-reading those two subjects before starting will save you hours.

Chapter IV

General guidelines

- The whole project has to be done in an **isolated environment**, and the agent itself runs inside a container. A single `docker compose up` must bring up the agent, the local LLM runtime, and the demo target service.
- All configuration and source files of your project go at the root of your repository, in the folders `p1`, `p2`, `p3`, and optionally `bonus`.
- The agent must be written in **Python** so peer-evaluation is straightforward. The frontend can be in any language.
- The language model has to run **locally**. The reference setup uses `Ollama` with a coder-oriented model of around 3B parameters; you are free to swap the runtime (`llama.cpp`, `vLLM`, plain `transformers`, etc.) and the model for anything else that fits on your laptop. The only hard rules: it must run on the host, and the parameter count must stay under 3 billion.
- The rest of the runtime stack is yours to choose. The reference layout is `FastAPI` plus a small HTML dashboard, brought up by `docker compose`; any equivalent that starts the whole system with one command and lets a reviewer drive it without reading source code is acceptable.
- The vector database is **not** a free choice: `ChromaDB` in `PersistentClient` mode is required. The index must be persistent across restarts, must update incrementally as files change (watch mode), and must **never** wipe and rebuild on every start. A peer who has already paid the indexing cost should not pay it again because the agent restarted.
- The agent talks to the host's Docker daemon. The reference layout mounts `/var/run/docker.sock` into the agent container; any equivalent that lets the agent inspect and restart sibling containers from inside its own container is acceptable.
- Pay close attention to file ownership on shared volumes. On Linux hosts, the agent container and the target may run as different UIDs. If the agent writes files as `root` and the target runs as an unprivileged user, `git` will refuse to operate on the repository and the target will fail to read its own source tree. Make sure the agent's `Dockerfile` sets a user whose UID matches the ownership of the target's source

directory, or use explicit volume permission settings (e.g. the `:z` flag for SELinux hosts).



The whole demo runs against a tiny target service of your own design. A FastAPI or Flask "hello" app of a few files is enough, as long as you can plant a bug that crashes it on start, and as long as the agent has read-write access to its source tree. Stick to explicit runtime failures the model can read straight from the log: a missing import, a bad environment variable, a syntax error. Business logic bugs are out of scope for a 3B model.

Chapter V

Mandatory part

The three parts below must be done in order.

V.1 Part 1: Observer

The agent's senses come first. The Observer watches the target from two angles and turns what it sees into a clean stream of events the rest of the pipeline can react to.

The container side reports the target's current state (running, restarting, exited), streams its standard output and standard error in near real time, and raises a *crash event* whenever the container exits with a non-zero status, enters a restart loop, or emits a line that matches a configurable list of error patterns. The HTTP side probes a configurable list of URLs on the target's public surface (typically the root and a healthcheck route). A connection refused, a timeout, or a 5xx response is treated as another flavour of crash event; a 4xx is a *suggestion* event, surfaced in the dashboard but never patched on its own.

A reference layout exposes all of this through HTTP routes (`GET /status`, `GET /logs`, `GET /events` over Server-Sent Events) and renders it in the dashboard's first tab. The exact route names are your choice; what matters is that a peer can see the target's health and read its recent logs without leaving the dashboard.

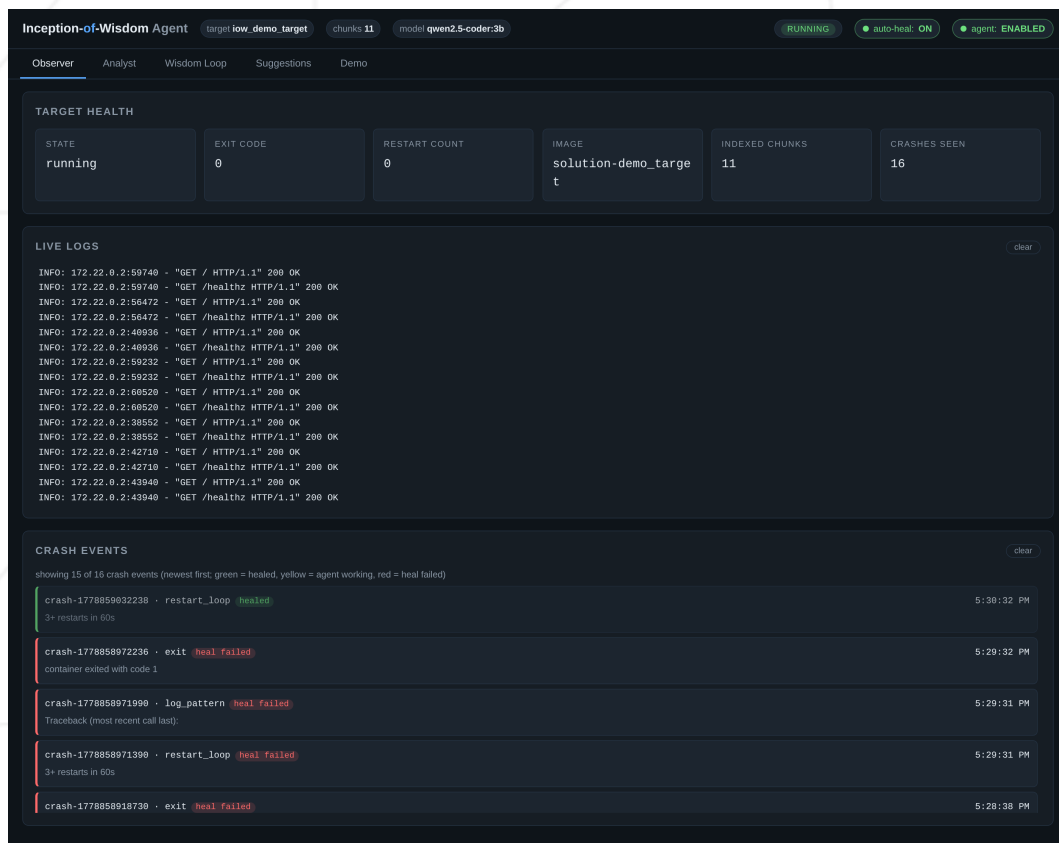


Figure V.1: *

Observer tab. Target health, live logs, and the deduplicated crash events the Observer has raised.

The split between hard and soft signals is what gives the agent its judgment. A real outage (page does not display, server died, container restart-looped) is something the wisdom loop should fix on its own. A functional gap (a missing route returning 404) belongs to a human; the agent flags it and waits.

Constraints:

- The Observer must work fully offline. It talks to the local Docker daemon and to the target's HTTP surface only.
- Crash detection has to deduplicate events. A single bad start should produce one event, not one per second. Suggestions follow the same discipline on a longer window so the same missing route does not flood the dashboard.
- The Observer must keep running after the target container goes away and recover when it comes back. A Docker daemon disconnect must not kill the agent.



Mounting `/var/run/docker.sock` into a container grants root-equivalent access to the host. Treat the agent's image as a privileged piece of software. Do not push it to a public registry without thinking through the trust model.



The docker Python SDK exposes `events()` and `logs(stream=True)` as generators. Wrap each one in a background task with its own retry loop, and forward the normalised events through an asyncio queue or a small SSE endpoint.

V.2 Part 2: Analyst

Sensing without understanding is useless. The Analyst turns a crash event into a structured diagnosis: which file is the most likely culprit, and why.

It builds and maintains a local **ChromaDB** index over the target service's source tree (the same indexing trick you used in *Inception-of-Context*: logical chunking, local embeddings, watch mode), then offers two operations on top of it. The first, retrieval, returns the top-*k* chunks closest to an arbitrary query, with their similarity score. The second, diagnosis, takes a crash event (status, log excerpt, exit code) and asks the local LLM to point at the file or files most likely responsible, and to explain in one paragraph what looks wrong.

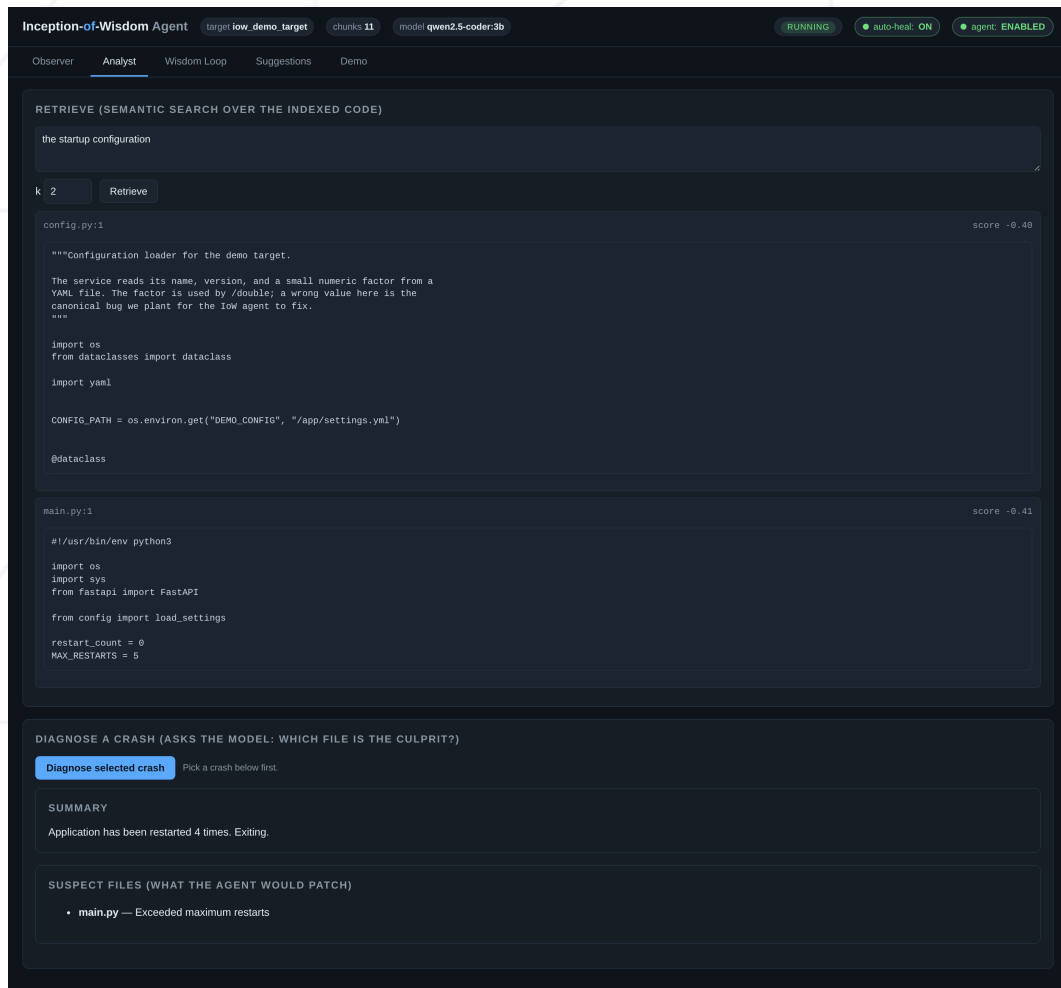


Figure V.2: *

Analyst tab. A free-text retrieval is shown at the top with its top-k chunks; below, a crash event is selected and the model's structured diagnosis names the suspect file with its reason.

The diagnosis output is structured (JSON, with at least a **summary** field and a **files** list of suspect paths). It is the input format that Part 3 expects.

Constraints:

- Re-use the indexing discipline from *Inception-of-Context*: logical chunks (not fixed-size), local embeddings, persistent storage, incremental watch mode.
- The diagnosis must cite real files from the index. It must not invent paths.
- If the model returns nothing usable (no suspect file, empty summary), the Analyst surfaces that as an explicit failure mode. The pipeline does not guess.



A small model will tell you a lot less than a frontier model for the same prompt. Use the index to do as much of the work as you can before reaching for the LLM: extract identifiers from the error log, look them up in the symbol list of the index, and feed only the relevant chunks to the model. The prompt should ask the model to choose, not to invent.

V.3 Part 3: Wisdom Loop

Now the agent has to actually fix things. Given a diagnosis, the loop generates a patch, applies it, commits, restarts the target, and decides whether the symptom is gone.

The patch format is the one you already know from *Inception-of-Context*: structured JSON, with a **summary** and a **files** array of {**path**, **op**, **content**} entries, where **content** is the complete post-change file and **op** is one of **create**, **modify**, **delete**. Diffs are not accepted.

Before any disk write, the patch has to pass the same kind of sanity checks as in IoC: refuse a patch that touches more than three files, replaces a non-empty file with a trivial placeholder, shrinks a file by more than 60%, or leaks retrieval markers into its content.

Once the sanity step is green, the loop runs the full sequence:

- Apply the patch atomically to the target's working tree.
- Make a Git commit on a dedicated branch (`iow/auto-heal` is a sensible default) with a short message generated from the diagnosis.
- Restart the target container.
- Observe the new state. The target is considered healed if it stays up for at least a configurable grace period and no new crash event fires.
- On failure, hand the new error log back to the Analyst and try once more, up to **three attempts**.

After three failed attempts, the Wisdom Loop rewinds the auto-heal branch to the pre-loop revision (the agent's commits become unreachable), restarts the target on the original code, and surfaces a clean failure to the dashboard.

A self-healing agent that runs an LLM on a laptop is two clicks away from cooking the host. The loop must ship with hard bounds on every level: one heal at a time (single-flight), a cooldown per crash signature so the same symptom cannot retrigger the loop on its own, a rate limit per hour, a hard cap over the lifetime of the process, and a global kill-switch the operator can flip to refuse every new heal. The default values are your choice; what matters is that a misbehaving demo cannot turn into a loop. All timing values (grace period, cooldown window, rate limit per hour, and hard cap) must be configurable through the `iow.config.yml` file at the target project root. Hard-coded timing values do not pass this part. A peer must be able to shorten the grace period to a few seconds for a smooth defense without touching the source code.

Inception-of-Wisdom (IoW)

Inception-of-Wisdom Agent

target low_demo_target chunks 11 model qwen2.5-coder:3b

RUNNING auto-heal: ON agent: ENABLED

Observer Analyst Wisdom Loop Suggestions Demo

FULL PIPELINE: "DIAGNOSE -- PATCH -- COMMIT -- RESTART -- VERIFY"

The Wisdom Loop takes a crash event and goes **all the way**: it asks the Analyst for a diagnosis, asks the model for a structured patch, runs sanity checks, writes the patch to disk, commits it on the auto-heal branch, restarts the target, and decides whether the symptom is gone. Up to **three attempts**; on the fourth failure, it rolls everything back through Git.

TRIGGER THE LOOP

Run heal loop on selected

Pick a crash below first.

showing 15 of 16 crash events (newest first; green = healed, yellow = agent working, red = heal failed)

crash-1778859032238 · restart_loop **healed**

3+ restarts in 60s

5:38:32 PM

crash-1778858972236 · exit **heal failed**

container exited with code 1

5:29:32 PM

crash-1778858971990 · log_pattern **heal failed**

Traceback (most recent call last):

5:29:31 PM

crash-1778858971390 · restart_loop **heal failed**

3+ restarts in 60s

5:29:31 PM

crash-1778858918730 · exit **heal failed**

5:28:38 PM

RESULT

HISTORY

showing 2 of 2 heal runs (click a row to expand)

5:31:25 PM · healed · 1 attempt · auto

a228d75 - d684c76

▼ collapse

attempt 1 — healed — target healthy after restart

diagnosis: The application has been restarted multiple times within a short period, which could indicate an issue with resource exhaustion or misconfiguration.

suspect files:

- main.py — The service is intentionally small and structured to be breakable in focused, reproducible ways.

commit: d684c76db

patch:

main.py (modify)

```
#!/usr/bin/env python3
- """A tiny FastAPI service used as the heal target for IoW.
+ import os
+ import sys
+ from fastapi import FastAPI
- The service is intentionally small and structured to be breakable in
- focused, reproducible ways. The IoW agent indexes this tree and tries
- to recover the service when one of those bugs is planted.
+ from config import load_settings
- import os
```

Figure V.3: *

Wisdom Loop tab. Each attempt is recorded: diagnosis, patch JSON, commit hash, post-restart health.

Rollback has to leave the repository in the exact state it was in before the loop started. The auto-heal branch may still exist, but the target's working tree and the deployed container must match the pre-loop revision byte for byte.

Snapshot the original commit hash before you start, and always restart the target from that hash on rollback. Do not try to manually undo writes by re-applying inverse patches; Git already knows how.

13

Chapter VI

Bonus part



The bonus part is graded only if the mandatory part is **perfect**. Perfect means the mandatory part has been done integrally and works without malfunctioning, even when the user pushes the system in weird directions. If you have not passed all the mandatory requirements, your bonus part will not be evaluated at all.

A few ideas, in no particular order:

- Open a Pull Request on a real remote (Gitea or a local GitHub mirror) instead of committing on a local branch, so a human can review the auto-heal before it lands.
- Plug a real GitOps controller (Argo CD against a k3d cluster, like in *Inception-of-Things*) so the commit alone triggers the redeploy.
- Train a tiny classifier on past events to skip the LLM entirely on known-symptom crashes.
- Add a "second opinion" mode where two prompts vote on the same diagnosis and only an agreement triggers the patch.
- A richer dashboard with the diff of the proposed patch, per-chunk relevance scores, and a timeline of past heals.

Chapter VII

Submission and peer-evaluation

Submit your work on your assigned `Git` repository. Only the code inside that repository will be graded during the defense. As usual, double-check that the official submission is the one being evaluated.

Your repository must contain, at its root, the folders `p1`, `p2`, and `p3`. A `bonus` folder may be added if the mandatory part is fully complete.

A `README.md` at the root has to describe how to start the system, and a `Makefile` at the root has to expose the entry points that bring it up and tear it down, for example `make up` backed by `docker compose up`. The peer must be able to run the project without reading the source.



Never commit a downloaded LLM weight, an embedding model archive, or a populated ChromaDB directory. These files are large, machine-specific, and useless to a reviewer. They belong in a `.gitignore`, not in the repository. A submission that pushes them is rejected on sight.